

Quality Audit

Developer Guide · digital QA 343-34

Technical notes for maintaining the app. Read it alongside the code; it covers the parts that are easy to get wrong.

Stack

- ASP.NET Core 8 Web API + EF Core 8, SQL Server (RittalQualityAudit on CSMSVR02).
- Single-page frontend: one static file, **wwwroot/index.html**. Vanilla HTML/CSS/JS, Chart.js from CDN. No npm, no build step, no framework.
- Deployed to IIS on csm-srv-16 (same pattern as the TL Portal).

Solution layout

Program.cs	host: DbContext, services, controllers, static files
appsettings.json	connection string + Storage:AttachmentRoot
Data/QualityAuditContext	EF model mapped onto the existing schema
Models/Entities.cs	tables
Models/Views.cs	keyless entities for the vw_* views
Models/Dtos.cs	request + response shapes
Services/WeekHelper.cs	Tuesday-week maths (mirror of dbo.fn_WeekStarting)
Services/SeverityMath.cs	resolve a machine's severity for a week
Services/UserContext.cs	current user + admin check (pluggable)
Services/AttachmentStorage	photos on disk
Controllers/	Departments, Form, Submissions, Attachments, Dashboard, Coverage, Admin
wwwroot/index.html	the entire frontend
sql/	one-off DB scripts to run in SSMS
docs/	this guide + the user guide

Run locally

```
dotnet run
```

Serves on `http://localhost:5000` (Properties/launchSettings.json). The frontend is static, so editing `wwwroot/index.html` only needs a browser refresh; C# changes need a rebuild.

Config keys in `appsettings.json`:

- **ConnectionStrings:RittalQualityAudit** – SQL Server connection.
- **Storage:AttachmentRoot** – folder for uploaded photos (a file share in production).

Build and deploy (IIS)

- Publish with Properties/PublishProfiles/FolderProfile.pubxml, or `dotnet publish -c Release`.
- Copy the output to the site on csm-srv-16; app pool = No Managed Code.
- The app pool identity needs read/write to Storage:AttachmentRoot and access to RittalQualityAudit on CSMSVR02.
- `index.html` is static, so a frontend-only change can be deployed by copying that one file. Browsers cache it – hard-refresh (Ctrl+F5) after deploying.

Database and EF mapping

The schema is hand-managed. Do not use EF migrations and do not create or alter tables from code. Entities map onto existing objects; a schema change is a `.sql` file in `sql/` to run in SSMS, plus a matching entity/DTO edit.

- Tables map by name. **Result.Outcome** maps to the SQL column **Result** via HasColumnName (a property can't share its class name).
- The six vw_* views are keyless entities (HasNoKey().ToView(...)), read-only.
- DateOnly maps to SQL date; Departments.TargetPercent is decimal (DECIMAL(5,2)).
- CreatedAt / SetAt / UploadedAt use DB defaults (ValueGeneratedOnAdd) – don't set them in code.

Tables: Departments, SeverityLevels, CheckPoints, AuditItems, SeverityAssignments, Customers, ActionTypes, NotAuditedReasons, AuditUsers, Submissions, Results, ResultCheckPoints, ResultAttachments.

Views: vw_CurrentSeverity, vw_WeeklyCompliance, vw_Failures, vw_FailuresByCustomer, vw_CheckPointFailures, vw_WeeklySummary. All dashboard views filter IsComplete = 1.

Domain rules that must not break

These are load-bearing. Getting them wrong corrupts reported figures.

- **The audit week starts on Tuesday.** WeekHelper.WeekStarting mirrors dbo.fn_WeekStarting (anchored on 1900-01-02). Submissions.WeekStarting is computed server-side on save; never trust a client value.
- **Severity is snapshotted.** Each result stores SeverityAtAudit, resolved for that week from SeverityAssignments, falling back to AuditItems.DefaultSeverity (SeverityMath.Resolve). Never join live severity onto historical results.
- **Machine identity is AuditItems.Id.** DisplayName is not unique. Order by SortOrder; show the row number. Never key on name.
- **NDT is per department.** The Destructive/NDT sub-check renders only when Departments.HasNdtCheck is true.
- **Result values are exactly OK, NOT_OK, NOT_AUDITED.** The UI shows OK / Not OK / No Time / Not Running; the last two store NOT_AUDITED with the NotAuditedReasons id whose Code is NO_TIME / NOT_RUNNING (looked up at runtime).
- **Mandatory fields** (client, server, and DB CHECK): any NOT_OK (row or sub-check) requires a Deviation; NOT_AUDITED requires a reason. A NOT_AUDITED row stores only its reason; other fields are nulled on save.
- **Attainment:** expected = sum over each week in the period of each active item's ChecksPerWeek for that week's severity; actual = results with OK or NOT_OK on complete submissions; target = Departments.TargetPercent. Nothing hardcoded.
- **IsComplete** separates a draft (0, resumable) from a submission (1). Only IsComplete = 1 feeds the dashboard views.

API surface

All JSON is camelCase; errors are { "error": "..." }.

- GET /api/departments
- GET /api/form/{departmentId}?date= – everything the entry form needs; severity resolved for that date's week
- POST /api/submissions – create draft or submit; returns id + auditItemId→resultId map
- PUT /api/submissions/{id} – update/resume (upsert by AuditItemId so photos survive)
- GET /api/submissions?from=&to=&departmentId=&shift=&search= and /api/submissions/{id}
- GET /api/submissions/draft?departmentId=&date=&shift= – resumable draft
- POST /api/results/{resultId}/attachments ; GET .../attachments ; GET /api/attachments/{id}
- GET /api/dashboard/attainment | summary | failures | by-customer | check-points | overview
- GET /api/coverage?departmentId=&weekStarting=
- GET/POST /api/admin/severities ; GET/POST/PUT /api/admin/audit-items | users | customers | check-points | action-types ; GET/PUT /api/admin/severity-levels | departments

Frontend

One file, wwwroot/index.html: a style block, the tab-pane markup, and one script. No modules.

- **Nothing is hardcoded.** loadForm() calls /api/form/{dept} and builds the machine list, RAG legend, dropdowns and check points from the response.
- State: **rows** is an object keyed by auditItemId. machineRowHtml / rowDetailHtml render a row; resultBankHtml renders the four buttons.
- Handlers: setResult (OK/Not OK; OK auto-fills sub-checks + check points), setNaResult (No Time/Not Running).
- validateRows returns {devBad, reasonBad}; save is blocked and rows highlighted until fixed.
- Draft safety net: localStorage key qa-draft-v3, debounced ~500 ms, cleared only after a confirmed 200. Server drafts (IsComplete = 0) are offered via the resume banner.
- Severity colours come from SeverityLevels.ColourHex; textOn(hex) picks readable text. Charts via Chart.js CDN.

Auth / admin seam

No authentication yet. Services/UserContext.cs reads an X-Username header and checks it against AuditUsers for IsAdmin; with no header it allows access (dev). When Windows Auth is added, only UserContext changes.

Common changes

- **Add/retire a machine, user, customer, check point, action type** – use the Admin tab or the /api/admin/* endpoints. Retire (soft delete), don't delete.
- **Change a colour, frequency, target or instruction** – Admin tab or a sql/ script. Data only; no redeploy.
- **Add an API endpoint** – add a method to the relevant controller (inject QualityAuditContext, AsNoTracking() for reads, project to a DTO). Admin endpoints start with the GuardAsync() check.
- **Schema change** – write a .sql in sql/, run it in SSMS, then add/adjust the entity (or keyless view) and the context mapping. No migration.

Gotchas

- The aggregate views have no date column, so date-ranged maths is computed from the base tables (SeverityMath + WeekHelper.WeeksBetween), not the views.
- After a frontend change, a stale index.html on the server or in the browser cache is the usual "it didn't update" cause – redeploy the file and Ctrl+F5.
- Photos need a persisted result first; the frontend saves a draft automatically before uploading so a resultId exists.